

**LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11**



Sultan Zulvi Risda

(105224034)

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA**

2026

1. Pendahuluan

Praktikum kali ini bertujuan untuk memahami dan mengimplementasikan mekanisme Error Handling (Penanganan Kesalahan) dalam pemrograman berbasis Java melalui studi kasus Sistem Reservasi Tiket Kereta Api "JAVA EXPRESS". Fokus utama praktikum ini adalah menjaga stabilitas aplikasi agar tidak mengalami henti mendadak (crash) saat menerima masukan yang tidak valid dari pengguna. Melalui modul ini, praktikan mempelajari cara memisahkan logika bisnis utama dengan logika penanganan kesalahan menggunakan struktur try-catch-finally. Selain itu, praktikum ini juga menekankan pada pembuatan Custom Exception baik yang bersifat Checked (wajib dideklarasikan atau ditangani secara lokal) maupun Unchecked (turunan dari RuntimeException) untuk mendefinisikan kondisi galat yang lebih spesifik dan relevan dengan domain aplikasi yang sedang dibangun.

2. Variabel

Nama Variabel	Tipe Data	Fungsi
kodeKereta	String	Menyimpan kode unik pengenalan kereta api
namaKereta	String	Menyimpan nama komersial dari armada kereta api
rute	String	Menyimpan informasi rute asal dan tujuan perjalanan kereta
sisakursi	int	Mencatat jumlah kapasitas kursi yang masih tersedia
daftarKereta	List<Kereta>	Menampung seluruh koleksi objek kereta di dalam memori
nik	String	Menyimpan Nomor Induk Kependudukan penumpang untuk validasi
namaPenumpang	String	Menyimpan nama lengkap dari calon penumpang
jumlahTiket	int	Menyimpan kuantitas tiket yang ingin dipesan dalam satu transaksi

berjalan	boolean	Menjadi penanda (<i>flag</i>) status aktif atau tidaknya perulangan menu utama
menu	int	Menampung nilai pilihan menu yang dimasukkan pengguna lewat CLI

3. Method dan Fungsi

Nama Method	Jenis Method	Fungsi
Kereta()	Konstruktor	Menginisialisasi nilai awal objek Kereta saat pertama kali dibuat
getKodeKereta()	Getter	Mengembalikan string kode unik milik objek kereta terkait
getNamaKereta()	Getter	Mengembalikan string nama dari objek kereta terkait
getRute()	Getter	Mengembalikan informasi rute perjalanan dari objek kereta terkait
getSisaKursi()	Getter	Mengembalikan jumlah sisa kursi yang tersedia pada objek kereta terkait
kurangiKursi(int)	Void	Mengurangi jumlah sisa kursi kereta berdasarkan kuantitas yang dipesan
ReservasiKontrol()	Konstruktor	Menginstansiasi list dan memicu otomatis pengisian data awal kereta
inisialisasiDataAwal()	Void	Memasukkan data armada kereta awal ke dalam daftar koleksi memori
getDaftarKereta()	Getter	Mengembalikan seluruh koleksi list objek kereta kepada pemanggil
pesanTiket()	Void	Memproses validasi ketat data transaksi dan

		melemparkan exception jika melanggar aturan bisnis
main(String[] args)	Static	Menjadi titik awal eksekusi program sekaligus mengontrol antarmuka CLI dan menangani semua exception.

4. Identifikasi Class Exception

- `DataPenumpangTidakValidException` (Unchecked Exception): Kelas ini mewakili kesalahan logika yang terjadi akibat kelalaian pengguna dalam memasukkan data. Kelas ini menginduk pada `RuntimeException` sehingga tidak wajib dideklarasikan menggunakan kata kunci `throws`.
- `RuteTidakDitemukanException` (Checked Exception): Kelas ini menangani kondisi di mana sistem tidak dapat menemukan objek kereta berdasarkan kode yang diinput. Karena menginduk pada kelas `Exception`, pemanggil metode wajib menangani objek ini.
- `TiketHabisException` (Checked Exception): Kelas ini dirancang khusus untuk menangani situasi di mana jumlah pesanan melampaui sisa kursi yang tersedia. Kelas ini bertipe checked dan membawa data internal berupa nama kereta serta sisa kursi aktual.

5. Penjelasan Program

```
public class DataPenumpangTidakValidException extends RuntimeException {
    public DataPenumpangTidakValidException(String pesan) {super(pesan);}
}
```

Kelas `DataPenumpangTidakValidException` membuat konstruktor kelas yang menerima parameter bertipe `String` dan meneruskan pesan kesalahan tersebut ke konstruktor induk (superclass) menggunakan kata kunci `super`

```
public class RuteTidakDitemukanException extends Exception {
    public RuteTidakDitemukanException(String pesan) {super(pesan);}
}
```

Kelas `RuteTidakDitemukanException` menyediakan konstruktor untuk menerima string pesan galat dan mengirimkannya ke objek induk

```
public class TiketHabisException extends Exception {
    private String namaKereta;
```

```

    private int sisaKursi;

    public TiketHabisException(String pesan, String namaKereta, int
sisaKursi) {

        super(pesan);

        this.namaKereta = namaKereta;

        this.sisaKursi = sisaKursi;

    }

    public String getNamaKereta() {

        return namaKereta;

    }

    public int getSisaKursi() {

        return sisaKursi;

    }

}

```

Kelas TiketHabisException

- Mendefinisikan variabel instans privat namaKereta dan sisaKursi untuk menyimpan konteks data ketika error terjadi
- Konstruktor yang menerima pesan error beserta data spesifik kereta, lalu mengikat nilai tersebut ke dalam atribut lokal
- Menyediakan method getter untuk mengakses informasi spesifik dari luar kelas saat proses penangkapan error (catch)

```

public class Kereta {

    private String kodeKereta;

    private String namaKereta;

    private String rute;

    private int sisaKursi;

```

Kelas Kereta mendefinisikan variabel kodeKereta, namaKereta, rute, dan sisaKursi secara enkapsulasi.

```

public Kereta(String kodeKereta, String namaKereta, String rute, int
sisaKursi) {

    this.kodeKereta = kodeKereta;

    this.namaKereta = namaKereta;

```

```
        this.rute = rute;

        this.sisaKursi = sisaKursi;
    }
}
```

Konstruktor utama untuk menetapkan nilai awal dari keempat atribut objek kereta menggunakan kata kunci this.

```
public String getKodeKereta() {

    return kodeKereta;

}

public String getNamaKereta() {

    return namaKereta;

}

public String getRute() {

    return rute;

}

public int getSisaKursi() {

    return sisaKursi;

}
}
```

Kumpulan method getter standar guna mengembalikan nilai dari masing-masing variabel enkapsulasi privat kelas Kereta

```
public void kurangiKursi(int jumlah) {

    this.sisaKursi -= jumlah;

}
}
```

Method yang berfungsi mengurangi nilai kapasitas kursi berdasarkan total tiket yang sukses dipesan oleh pelanggan

```
import java.util.ArrayList;

import java.util.List;
```

```
public class ReservasiKontrol {

    private List<Kereta> daftarKereta;
```

Kelas ReservasiKontrol

- Mendeklarasikan kelas ReservasiKontrol yang mengatur jalur pengolahan data transaksi
- Membuat variabel penampung berbasis koleksi objek List dengan tipe generik Kereta

```
public ReservasiKontrol() {

    daftarKereta = new ArrayList<>();

    inisialisasiDataAwal();

}
```

Konstruktor yang bertugas mengalokasikan objek ArrayList ke dalam variabel daftarKereta dan memanggil method pengisian data

```
private void inisialisasiDataAwal() {

    daftarKereta.add(new Kereta("K01", "Argo Bromo", "JKT - SBY",
50));

    daftarKereta.add(new Kereta("K02", "Parahyangan", "JKT - BDG",
15));

}

public List<Kereta> getDaftarKereta() {

    return daftarKereta;

}
```

- Memasukkan dua objek armada awal kereta api secara otomatis ke dalam memori sistem sesuai instruksi soal tugas
- Menyediakan akses keluar bagi kelas lain untuk membaca isi koleksi list kereta

```
public void pesanTiket(String kodeKereta, String nik, String
namaPenumpang, int jumlahTiket)

    throws RuteTidakDitemukanException, TiketHabisException {
```

Mendeklarasikan metode pesanTiket yang secara eksplisit mendelegasikan tanggung jawab penanganan dua objek checked exception (RuteTidakDitemukanException dan TiketHabisException) kepada pihak pemanggil menggunakan instruksi throws

```
if (nik.length() != 16 || !nik.matches("\\d+")) {  
    throw new DataPenumpangTidakValidException("NIK harus tepat  
16 digit angka.");  
}
```

- Melakukan validasi kesesuaian data NIK melalui pengecekan panjang karakter dan pola ekspresi reguler angka (\d+)
- Melempar objek instans baru dari DataPenumpangTidakValidException menggunakan kata kunci throw apabila validasi NIK gagal memenuhi kriteria.

```
Kereta keretaDipilih = null;  
  
for (Kereta k : daftarKereta) {  
    if (k.getKodeKereta().equalsIgnoreCase(kodeKereta)) {  
        keretaDipilih = k;  
        break;  
    }  
}
```

- Menyiapkan variabel lokal keretaDipilih dengan status awal kosong (null)
- Melakukan iterasi perulangan for-each untuk mencari keberadaan kereta di dalam list berdasarkan kesamaan kode input

```
if (keretaDipilih == null) {  
    throw new RuteTidakDitemukanException("Kereta dengan kode  
'" + kodeKereta + "' tidak ditemukan.");  
}
```

Jika setelah perulangan nilai variabel pencarian tetap bernilai null, program secara instan memicu lompatan error dengan melemparkan RuteTidakDitemukanException

```
if (jumlahTiket > keretaDipilih.getSisaKursi()) {  
    throw new TiketHabisException(  
        "Jumlah pemesanan > sisa kursi yang tersedia.",  
        keretaDipilih.getNamaKereta(),  
        keretaDipilih.getSisaKursi());  
}
```


- Memvalidasi ketersediaan kapasitas kursi terhadap jumlah pemesanan tiket yang diajukan
- Jika permintaan melebihi kuota, program melemparkan TiketHabisException dengan menyertakan pesan, nama kereta, beserta sisa kursi aktual yang tersedia

```

keretaDipilih.kurangiKursi(jumlahTiket);

    System.out.println("Pemesanan Berhasil!");

    System.out.println("Penumpang: " + namaPenumpang);

    System.out.println("Kereta: " + keretaDipilih.getNamaKereta() +
" (" + jumlahTiket + " tiket)");

    }

}

```

- Memperbarui data sisa kursi di memori dengan memanggil metode pengurang kursi internal objek terkait
- Menampilkan notifikasi laporan ringkas keberhasilan transaksi pemesanan tiket kepada pengguna

```

import java.util.InputMismatchException;

import java.util.Scanner;

public class App {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        ReservasiKontrol kontrol = new ReservasiKontrol();

        boolean berjalan = true;

        System.out.println("SISTEM RESERVASI TIKET KERETA API JAVA
EXPRESS");

```

Kelas Main

- Menginstansiasi objek pembaca input keyboard, objek pusat kontrol reservasi, serta menetapkan status awal perulangan sistem
- Menampilkan judul aplikasi saat program pertama kali dimuat

```

try {

    while (berjalan) {

        System.out.println("\nMENU UTAMA");

```

```

        System.out.println("1. Lihat Jadwal & Sisa Kursi");

        System.out.println("2. Pesan Tiket");

        System.out.println("3. Keluar");

        System.out.print("Pilih menu: ");

```

Memulai perulangan bersyarat untuk menyajikan menu antarmuka secara terus-menerus selama nilai variabel kontrol bernilai benar dengan mencetak pilihan menu utama sistem reservasi ke layar terminal

```

int menu = 0;

        try {

            menu = scanner.nextInt();

            scanner.nextLine();

        } catch (InputMismatchException e) {

            System.out.println("Error: Pilihan menu harus
berupa angka!");

            scanner.nextLine();

            continue;

        }

```

- Menginisialisasi variabel lokal penampung pilihan menu
- Membuka blok pelindung try lokal untuk mengisolasi proses pembacaan angka menu
- Membaca data angka pilihan pengguna dan langsung membersihkan sisa karakter enter (newline) yang tertinggal di dalam objek buffer memori
- Blok catch mendeteksi dan menangkap galat InputMismatchException jika pengguna keliru memasukkan data non-angka
- Menampilkan pesan edukasi kesalahan, membersihkan tumpukan sisa string ilegal dari memori agar terhindar dari infinite loop, dan memaksa sistem melompat langsung menuju awal iterasi perulangan menu berikutnya lewat perintah continue

```

switch (menu) {

        case 1:

            System.out.println("\nJADWAL KERETA API:");

            for (Kereta k : kontrol.getDaftarKereta()) {

                System.out.println(k.getKodeKereta() + " |
" + k.getNamaKereta() + " | " + k.getRute() + " | Sisa Kursi: " +
k.getSisaKursi());

            }

```

```
break;
```

- Membuka percabangan multi-kondisi berdasarkan nilai dari variabel nomor menu
- Penanganan untuk menu 1, melakukan ekstraksi seluruh isi daftar koleksi kereta api dan mencetaknya ke layar, diakhiri dengan instruksi lompatan keluar break

```
case 2:

    try {

        System.out.print("Masukkan Kode Kereta: ");

        String kode = scanner.nextLine();

        System.out.print("Masukkan NIK Penumpang:");

        String nik = scanner.nextLine();

        System.out.print("Masukkan Nama Penumpang:");

        String nama = scanner.nextLine();

        System.out.print("Masukkan Jumlah Tiket:");

        int jumlahTiket = scanner.nextInt();

        scanner.nextLine();

        kontrol.pesanTiket(kode, nik, nama, jumlahTiket);
```

- Penanganan untuk menu 2 (Proses Reservasi)
- Membuka blok pengaman try tingkat transaksi untuk menampung seluruh potensi kegagalan input maupun pelanggaran aturan bisnis pemesanan
- Menampilkan teks panduan isian dan membaca input teks berupa kode, nomor NIK, serta nama penumpang
- Membaca input kuantitas pembelian tiket dan mengosongkan kembali sisa karakter baris baru di dalam buffer penampung scanner
- Mengirimkan seluruh variabel input ke metode milik pusat kendali logis reservasi tiket

```
} catch (InputMismatchException e) {

    System.out.println("Error: Jumlah tiket harus dimasukkan dalam bentuk angka!");

    scanner.nextLine();
```

```

        } catch (DataPenumpangTidakValidException e) {
            System.out.println("Kegagalan Validasi: " +
e.getMessage());

        } catch (RuteTidakDitemukanException e) {
            System.out.println("Gagal Memesan: " +
e.getMessage());

        } catch (TiketHabisException e) {
            System.out.println("Gagal Memesan: " +
e.getMessage());

            System.out.println("Detail: Kereta " +
e.getNamaKereta() + " hanya tersisa " + e.getSisaKursi() + " kursi.");
        }

        break;

```

- Menangkap galat salah tipe input jumlah tiket, mencetak peringatan, dan menyapu bersih data sampah yang tertinggal di buffer scanner
- Menangkap objek penanda kesalahan isian NIK (Unchecked) lalu mencetak pesan deskripsi galat bawaannya
- Menangkap objek penanda ketiadaan rute kereta (Checked) lalu memberikan informasi penolakan pesanan
- Menangkap objek error saat kursi habis (Checked), lalu memanfaatkan fungsi getter khusus miliknya untuk mengekstrak dan menampilkan detail nama armada beserta sisa kursi secara informatif kepada pembaca
- Menutup batasan ruang lingkup pemrosesan menu kedua

```

case 3:

        berjalan = false;

        break;

        default:

            System.out.println("Pilihan menu tidak
tersedia. Silakan pilih menu 1-3.");

        }

    }

```

- Penanganan menu 3, mengubah status kondisi perulangan utama menjadi salah agar siklus putaran menu terhenti, diikuti perintah penutup break

- Bagian cadangan untuk memproses masukan angka di luar jangkauan pilihan operasional sistem
- Penutup kurung kurawal pembatas perulangan while

```

} finally {

        System.out.println("\nTerima kasih telah menggunakan
layanan JAVA EXPRESS.");

        scanner.close();

    }

}

```

- Membuka blok perintah penutup finally. Blok ini dijamin oleh mesin Java akan selalu dieksekusi dalam kondisi apa pun, baik saat program berjalan lancar, terjadi error di tengah jalan, maupun ketika pengguna memilih menu keluar secara normal
- Mencetak salam penutup aplikasi ke terminal

6. Kesimpulan

Praktikum ini memberikan pemahaman mendalam mengenai pentingnya arsitektur keamanan kode program melalui teknik Exception Handling. Melalui simulasi sistem reservasi "JAVA EXPRESS", dapat disimpulkan bahwa penerapan blok pelindung try-catch yang spesifik sangat efektif dalam menyaring berbagai macam kegagalan sistem, baik yang disebabkan oleh ketidaksesuaian tipe data bawaan (InputMismatchException) maupun pelanggaran aturan bisnis yang sengaja dirancang lewat custom exception terstruktur (DataPenumpangTidakValidException, RuteTidakDitemukanException, dan TiketHabisException). Penggunaan kata kunci throws terbukti membantu pengembang dalam mendistribusikan tanggung jawab penanganan error dari level bawah (logika bisnis) menuju level atas (antarmuka program). Terakhir, fungsi blok finally terkonfirmasi sebagai fondasi krusial untuk menjamin eksekusi akhir pembersihan memori atau penutupan aliran data secara aman tanpa terpengaruh oleh interupsi galat yang terjadi sebelumnya.

7. Daftar Pustaka

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). Modul Praktikum Pemrograman Berorientasi Objek: Java Collections. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.